Alternative Assessment Tool (AAT) Report on

“Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL”

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Shreelaxmi Sangam 1BM24EC194

Submitted to

Prof. Ajaykumar Devarapalli

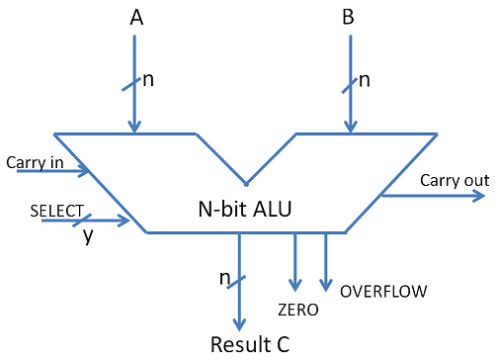
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019
Academic Year

2026 – 2027

1) Arithmetic Logic Unit (ALU)

DESIGN:



S_3	S_2	S_1	S_0	Operation	Function
0	0	0	0	A + B	Addition
0	0	0	1	A – B	Subtraction
0	0	1	0	A + 1	Increment A
0	0	1	1	A – 1	Decrement A
0	1	0	0	B + 1	Increment B
0	1	0	1	B – 1	Decrement B
0	1	1	0	A	Transfer A
0	1	1	1	A * B	Multiplication
1	0	0	0	A`	Compliment A
1	0	0	1	B`	Compliment B
1	0	1	0	A and B	AND
1	0	1	1	A or B	OR
1	1	0	0	A nand B	NAND
1	1	0	1	A nor B	NOR
1	1	1	0	A xor B	XOR
1	1	1	1	A xnor B	XNOR

VERILOG CODE:

```
module ALU (
    input [3:0] A, B,
    input [2:0] sel,
    output reg [3:0] Y
);

always @(*) begin
    case (sel)
        3'b000: Y = A + B;         // Addition
        3'b001: Y = A - B;         // Subtraction
        3'b010: Y = A & B;         // AND
        3'b011: Y = A | B;         // OR
        3'b100: Y = A ^ B;         // XOR
        3'b101: Y = ~A;            // NOT A
        3'b110: Y = A << 1;        // Left shift
        3'b111: Y = A >> 1;        // Right shift
        default: Y = 4'b0000;
    endcase
end

endmodule
```

TESTBENCH:

```
module ALU_tb;
```

```
reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;

ALU uut (
    .A(A),
    .B(B),
    .sel(sel),
    .Y(Y)
);

initial begin
    A = 4'b0101;
    B = 4'b0011;

    sel = 3'b000; #10; // Addition
    sel = 3'b001; #10; // Subtraction
    sel = 3'b010; #10; // AND
    sel = 3'b011; #10; // OR
    sel = 3'b100; #10; // XOR
    sel = 3'b101; #10; // NOT
    sel = 3'b110; #10; // Left shift
    sel = 3'b111; #10; // Right shift

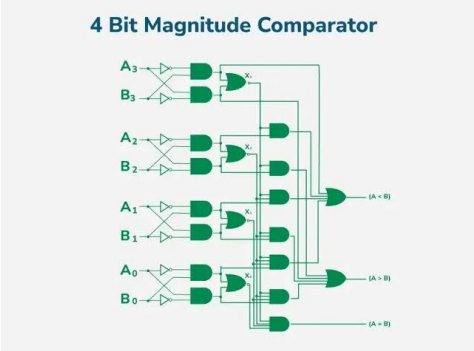
    $finish;
end

initial begin
    $monitor("Time=%0t A=%b B=%b sel=%b Y=%b",
             $time, A, B, sel, Y);
end

endmodule
```

2) Comparator

DESIGN:



INPUT				OUTPUT		
A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

VERILOG CODE:

```
module comparator (
   input [3:0] A, B,
   output reg gt, lt, eq
);

always @(*) begin
   if (A > B) begin
      gt = 1;
      lt = 0;
      eq = 0;
   end
   else if (A < B) begin
      gt = 0;
      lt = 1;
      eq = 0;
   end
   else begin
      gt = 0;
      lt = 0;
      eq = 1;
   end
end
```

```
endmodule
```

TESTBENCH:

```
module comparator_tb;

reg [3:0] A, B;
wire gt, lt, eq;

comparator uut (
    .A(A),
    .B(B),
    .gt(gt),
    .lt(lt),
    .eq(eq)
);

initial begin
    A = 4'd9; B = 4'd5; #10;
    A = 4'd3; B = 4'd7; #10;
    A = 4'd6; B = 4'd6; #10;
    A = 4'd2; B = 4'd8; #10;

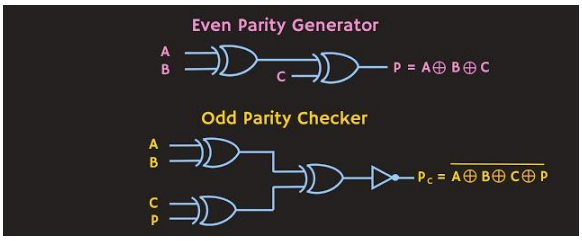
    $finish;
end

initial begin
    $monitor("Time=%0t A=%d B=%d GT=%b LT=%b EQ=%b",
            $time, A, B, gt, lt, eq);
end

endmodule
```

3) 4-Bit Parity Generator and Checker

DESIGN:



4-bit message				Even Parity Generator
X1	X2	X3	P	O
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

VERILOG HDL CODE:

```
module parity_generator (
    input [3:0] A,
    output even_parity,
    output odd_parity
);

assign even_parity = ^A;
assign odd_parity  = ~(^A);

endmodule

module parity_checker (
    input [3:0] A,
    input parity_bit,
    output even_error,
    output odd_error
);

assign even_error = ^{A, parity_bit};
assign odd_error  = ~(^({A, parity_bit}));

endmodule
```

TESTBENCH:

```
module parity_tb;
```

```
reg [3:0] A;
reg parity_bit;

wire even_parity, odd_parity;
wire even_error, odd_error;

parity_generator G1 (
   .A(A),
   .even_parity(even_parity),
   .odd_parity(odd_parity)
);

parity_checker C1 (
   .A(A),
   .parity_bit(parity_bit),
   .even_error(even_error),
   .odd_error(odd_error)
);

initial begin
   A = 4'b0000; parity_bit = 0; #10;
   A = 4'b0001; parity_bit = 1; #10;
   A = 4'b0011; parity_bit = 0; #10;
   A = 4'b0111; parity_bit = 1; #10;
   A = 4'b1111; parity_bit = 0; #10;

   $finish;
end
initial begin
   $monitor("Time=%0t A=%b P=%b EP=%b OP=%b EE=%b OE=%b",
      $time, A, parity_bit, even_parity, odd_parity,
      even_error, odd_error);
end
endmodule
```

4) 4-Bit Barrel Shifter

DESIGN:

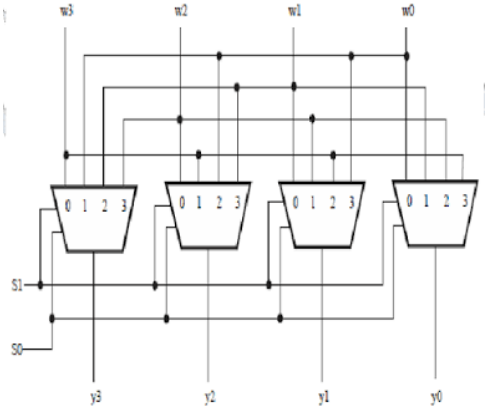


Table 1

	DS	DD1	DD0	A0	A1	A2	A3	operation
	0	0	0	1	0	0	0	through
left	0	0	1	0	1	0	0	left 1-bit shift
	0	1	0	0	0	1	0	left 2-bit shift
	0	1	1	0	0	0	1	left 3-bit shift
	1	0	0	0	0	0	1	through
right	1	0	1	0	0	1	0	right 1-bit shift
	1	1	0	0	1	0	0	right 2-bit shift
	1	1	1	1	0	0	0	right 3-bit shift

VERILOG HDL CODE:

```
module barrel_shifter (
    input [3:0] A,
    input [1:0] shamt,
    input dir,
    output reg [3:0] Y
);

always @(*) begin
    case (dir)
        1'b0: Y = A << shamt;  // Left shift
        1'b1: Y = A >> shamt;  // Right shift
        default: Y = A;
    endcase
end

endmodule
```

TESTBENCH:

```
module barrel_shifter_tb;
```

```
reg [3:0] A;
reg [1:0] shamt;
reg dir;
wire [3:0] Y;

barrel_shifter uut (
    .A(A),
    .shamt(shamt),
    .dir(dir),
    .Y(Y)
);

initial begin
    // Left shift tests
    A = 4'b1011; dir = 0; shamt = 0; #10;
    A = 4'b1011; dir = 0; shamt = 1; #10;
    A = 4'b1011; dir = 0; shamt = 2; #10;
    A = 4'b1011; dir = 0; shamt = 3; #10;

    // Right shift tests
    A = 4'b1011; dir = 1; shamt = 0; #10;
    A = 4'b1011; dir = 1; shamt = 1; #10;
    A = 4'b1011; dir = 1; shamt = 2; #10;
    A = 4'b1011; dir = 1; shamt = 3; #10;

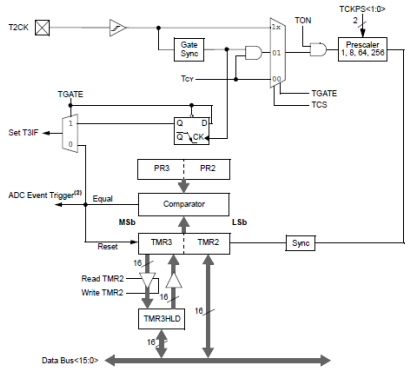
    $finish;
end

initial begin
    $monitor("Time=%0t A=%b dir=%b shamt=%d Y=%b",
             $time, A, dir, shamt, Y);
end

endmodule
```

5) Timer Module

DESIGN:



Input			Output	
D	reset	clock	Q	Q'
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	1	0
1	1	0	0	1
1	1	1	0	1

VERILOG HDL CODE:

```
module timer (
    input clk,
    input rst,
    input start,
    output reg done
);

reg [3:0] count;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        count <= 0;
        done  <= 0;
    end
    else if (start) begin
        if (count == 4'd9) begin
            count <= 0;
            done  <= 1;
        end
        else begin
```

```
            count <= count + 1;
            done  <= 0;
        end
    end
    else begin
        count <= 0;
        done  <= 0;
    end
end

endmodule
```

TESTBENCH:

```
module timer_tb;

reg clk, rst, start;
wire done;

timer uut (
    .clk(clk),
    .rst(rst),
    .start(start),
    .done(done)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk   = 0;
    rst   = 1;
    start = 0;

    #10 rst = 0;
    #10 start = 1;
```

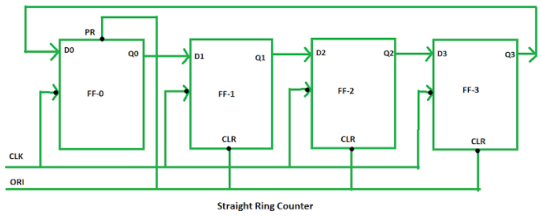
```
    #120 start = 0;
    #20 $finish;
end

initial begin
    $monitor("Time=%0t rst=%b start=%b done=%b",
            $time, rst, start, done);
end

endmodule
```

6) Ring Counter

DESIGN:



Straight Ring Counter

Clock Input	Q3	Q2	Q1	Q0
1	0	0	0	1
2	0	0	1	0
3	0	1	0	0
4	1	0	0	0
5	0	0	0	1
6	0	0	1	0
7	0	1	0	0
8	1	0	0	0

VERILOG HDL CODE:

```
module ring_counter (
    input clk,
    input rst,
    output reg [3:0] Q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        Q <= 4'b0001;
    else
        Q <= {Q[2:0], Q[3]};
end
```

```
endmodule
```

TESTBENCH:

```
module ring_counter_tb;

reg clk, rst;
wire [3:0] Q;

ring_counter uut (
    .clk(clk),
    .rst(rst),
    .Q(Q)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

    #10 rst = 0;

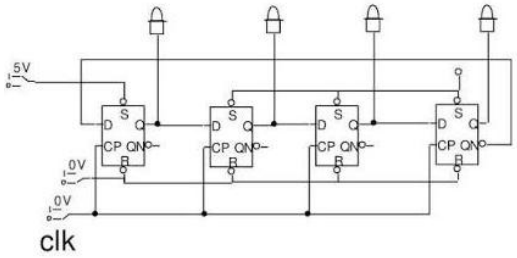
    #60 $finish;
end

initial begin
    $monitor("Time=%0t rst=%b Q=%b",
            $time, rst, Q);
end

endmodule
```

7) Johnson Counter

DESIGN:



Clock Input	Q3	Q2	Q1	Q0
1	0	0	0	1
2	0	0	1	0
3	0	1	0	0
4	1	0	0	0
5	0	0	0	1
6	0	0	1	0
7	0	1	0	0
8	1	0	0	0

VERILOG HDL CODE:

```
module johnson_counter (
    input clk,
    input rst,
    output reg [3:0] Q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], ~Q[3]};
end

endmodule
```

TESTBENCH:

```
module johnson_counter_tb;

reg clk, rst;
wire [3:0] Q;

johnson_counter uut (
    .clk(clk),
    .rst(rst),
```

```
    .Q(Q)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

    #10 rst = 0;

    #90 $finish;
end

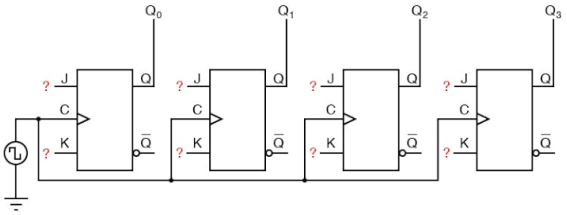
initial begin
    $monitor("Time=%0t rst=%b Q=%b",
            $time, rst, Q);
end

endmodule
```

8) Synchronous and Asynchronous Counters

➢ Synchronous Counter

DESIGN:



State	Q_D	Q_C	Q_B	Q_A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1
0	0	0	0	0

VERILOG HDL CODE:

```
module sync_counter (
    input clk,
    input rst,
    output reg [3:0] Q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        Q <= 4'b0000;
    else
        Q <= Q + 1'b1;
end

endmodule
```

TESTBENCH:

```
module sync_counter_tb;

reg clk, rst;
wire [3:0] Q;

sync_counter uut (
    .clk(clk),
```

```
    .rst(rst),
    .Q(Q)
);

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

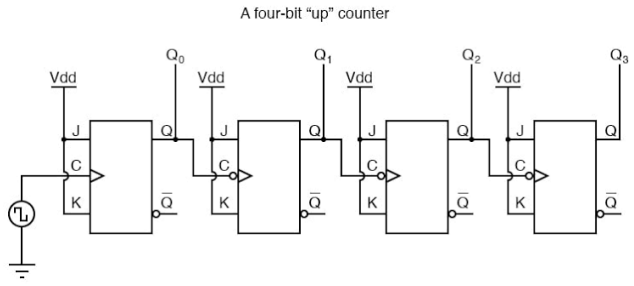
    #10 rst = 0;
    #160 $finish;
end

initial begin
    $monitor("Time=%0t rst=%b Q=%b",
            $time, rst, Q);
end

endmodule
```

- **Asynchronous Counter**

DESIGN:



Clock Cycles	Output Bits				Decimal Value
	QA	QB	QC	QD	
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	1	0	2
4	0	0	1	1	3
5	0	1	0	0	4
6	0	1	0	1	5
7	0	1	1	0	6
8	0	1	1	1	7
9	1	0	0	0	8
10	1	0	0	1	9
11	Counter Output Reset to Zero				

VERILOG HDL CODE:

```
module async_counter (
    input clk,
    input rst,
```

```
    output reg [3:0] Q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        Q[0] <= 0;
    else
        Q[0] <= ~Q[0];
end

always @(negedge Q[0] or posedge rst) begin
    if (rst)
        Q[1] <= 0;
    else
        Q[1] <= ~Q[1];
end

always @(negedge Q[1] or posedge rst) begin
    if (rst)
        Q[2] <= 0;
    else
        Q[2] <= ~Q[2];
end

always @(negedge Q[2] or posedge rst) begin
    if (rst)
        Q[3] <= 0;
    else
        Q[3] <= ~Q[3];
end

endmodule
```

TESTBENCH:

```
module async_counter_tb;
```

```
reg clk, rst;
wire [3:0] Q;

async_counter uut (
    .clk(clk),
    .rst(rst),
    .Q(Q)
);

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

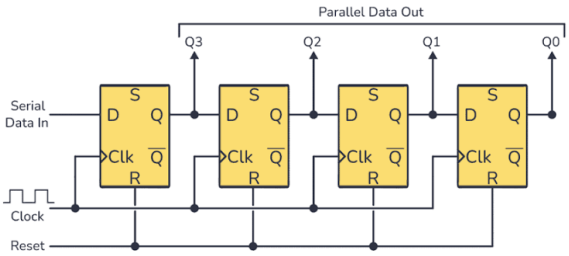
    #12 rst = 0;
    #200 $finish;
end

initial begin
    $monitor("Time=%0t rst=%b Q=%b",
            $time, rst, Q);
end

endmodule
```

9) Shift Register

DESIGN:



Clock Pulse	Q1	Q2	Q3	Q4
0	0	0	0	1
1	0	0	0	0
2	1	0	0	0
3	1	1	0	0
4	1	1	1	0
5	1	1	1	1
6	0	1	1	1
7	0	0	1	1

VERILOG HDL CODE:

```
module shift_register (
    input clk,
    input rst,
    input serial_in,
    output reg [3:0] Q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        Q <= 4'b0000;
    else
        Q <= {Q[2:0], serial_in};
end

endmodule
```

TESTBENCH:

```
module shift_register_tb;

reg clk, rst, serial_in;
wire [3:0] Q;

shift_register uut (
```

```
    .clk(clk),
    .rst(rst),
    .serial_in(serial_in),
    .Q(Q)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    serial_in = 0;

    #10 rst = 0;

    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;

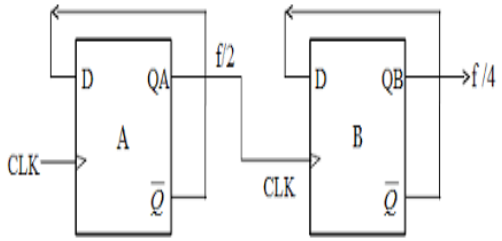
    $finish;
end

initial begin
    $monitor("Time=%0t rst=%b serial_in=%b Q=%b",
            $time, rst, serial_in, Q);
end

endmodule
```

10) Divide-by-N Counter

DESIGN:



Clock Pulse	Q_C	Q_B	Q_A
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0
9	0	0	1
10	0	0	0

VERILOG HDL CODE:

```
module divide_by_n (
   input clk,
   input rst,
   output reg clk_out
);

reg [3:0] count;

always @(posedge clk or posedge rst) begin
   if (rst) begin
      count   <= 0;
      clk_out <= 0;
   end
   else begin
      if (count == 4'd4) begin
         count   <= 0;
         clk_out <= ~clk_out;
      end
      else begin
         count <= count + 1'b1;
      end
   end
end
```

```
endmodule
```

TESTBENCH:

```
module divide_by_n_tb;

reg clk, rst;
wire clk_out;

divide_by_n uut (
    .clk(clk),
    .rst(rst),
    .clk_out(clk_out)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

    #12 rst = 0;

    #220 $finish;
end

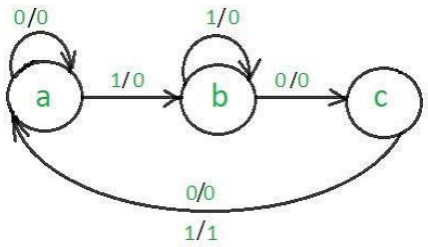
initial begin
    $monitor("Time=%0t rst=%b clk_out=%b",
            $time, rst, clk_out);
end

endmodule
```

11) Mealy and Moore FSMs

- **Mealy FSM**

DESIGN:



Present States		i/p	Next States		Flip Flop Excitations		O/P
X	Y		X'	Y'	Dx	Dy	
0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	0	0
0	1	1	0	0	0	0	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	X	X	X	X	X
1	1	1	X	X	X	X	X

VERILOG HDL CODE:

```
module mealy_101 (
   input clk, rst, x,
   output reg z
);
   reg [1:0] state, next_state;

   parameter S0 = 2'b00,
             S1 = 2'b01,
             S2 = 2'b10;

   always @(posedge clk or posedge rst) begin
      if (rst)
         state <= S0;
      else
         state <= next_state;
   end

   always @(*) begin
      next_state = state;
      z = 0;
```

```
        case (state)
            S0: begin
                if (x) next_state = S1;
                else   next_state = S0;
            end

            S1: begin
                if (x) next_state = S1;
                else   next_state = S2;
            end

            S2: begin
                if (x) begin
                    next_state = S1;
                    z = 1;
                end
                else
                    next_state = S0;
            end

            default: next_state = S0;
        endcase
    end
endmodule
```

TESTBENCH:

```
module tb_mealy_101;
    reg clk, rst, x;
    wire z;

    mealy_101 uut (
        .clk(clk),
        .rst(rst),
        .x(x),
        .z(z)
```

```
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        x = 0;

        #10 rst = 0;

        // Input sequence: 101
        #10 x = 1;
        #10 x = 0;
        #10 x = 1;

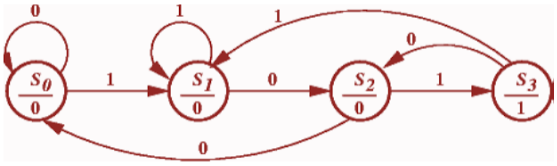
        // Continue input sequence
        #10 x = 0;
        #10 x = 1;
        #10 x = 1;
        #10 x = 0;

        #10 $finish;
    end

    initial begin
        $monitor("Time=%0t x=%b z=%b",
                 $time, x, z);
    end
endmodule
```

- **Moore FSM**

DESIGN:



VERILOG HDL CODE:

```
module moore_101 (
   input clk, rst, x,
   output reg z
);
   reg [1:0] state, next_state;

   parameter S0 = 2'b00,
             S1 = 2'b01,
             S2 = 2'b10,
             S3 = 2'b11;

   always @(posedge clk or posedge rst) begin
      if (rst)
         state <= S0;
      else
         state <= next_state;
   end

   always @(*) begin
      next_state = state;

      case (state)
         S0: begin
            if (x) next_state = S1;
            else   next_state = S0;
         end
```

```
            S1: begin
                if (x) next_state = S1;
                else   next_state = S2;
            end

            S2: begin
                if (x) next_state = S3;
                else   next_state = S0;
            end

            S3: begin
                if (x) next_state = S1;
                else   next_state = S2;
            end

            default: next_state = S0;
        endcase
    end

    always @(*) begin
        case (state)
            S3:     z = 1;
            default: z = 0;
        endcase
    end
endmodule
```

TESTBENCH:

```
`timescale 1ns/1ps

module tb_moore_101;
    reg clk, rst, x;
    wire z;
```

```
    moore_101 uut (
        .clk(clk),
        .rst(rst),
        .x(x),
        .z(z)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        x = 0;

        #10 rst = 0;

        // Input sequence: 101
        #10 x = 1;
        #10 x = 0;
        #10 x = 1;

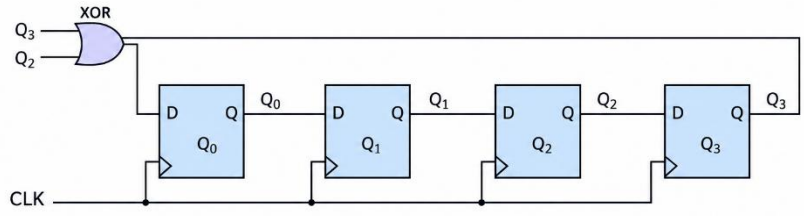
        // Continue input sequence
        #10 x = 0;
        #10 x = 1;
        #10 x = 1;
        #10 x = 0;

        #10 $finish;
    end

    initial begin
        $monitor("Time=%0t x=%b z=%b",
                 $time, x, z);
    end
endmodule
```

12) Random Sequence Generator

DESIGN:



Example sequence (decimal)

Clock cycle	Q_3	Q_2	Q_1	Q_0	Decimal
0 (reset)	0	0	0	1	1
1	0	0	1	1	3
2	0	1	1	0	6
3	1	1	0	0	12
4	1	0	0	1	9
5	0	0	1	0	2
...	...	...	...	...	...

VERILOG HDL CODE:

```
module random_seq (
    input clk, rst,
    output reg [3:0] Q
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            Q <= 4'b0001;
        else
            Q <= {Q[2:0], Q[3] ^ Q[2]};
    end

endmodule
```

TESTBENCH:

```
`timescale 1ns/1ps

module tb_random_seq;
    reg clk, rst;
    wire [3:0] Q;

    random_seq uut (
        .clk(clk),
        .rst(rst),
```

```
        .Q(Q)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;

        #10 rst = 0;

        #200 $finish;
    end

    initial begin
        $monitor("Time=%0t Q=%b", $time, Q);
    end
endmodule
```